

Entry 1 – What is a Finance Assistant App?

Artifact:



what is a finance assistant app

A finance assistant app is a tool that helps you track and understand your money. It typically:

- Shows you where your money goes each month
- Helps you set and reach savings goals
- Gives advice on how to spend less
- Uses AI to answer your finance questions

Think of it like a smart calculator that also gives you personalised advice.



can you connect this to my specific project, the user spends money on xbox gaming, gym, cinema at MCine Moka, and tea at Maison de Thé in Moka

For your project specifically, the Smart Finance Assistant helps a young person who spends on:

- Xbox games and DLC on the Xbox Store
- Gym membership and personal training sessions
- Cinema tickets at MCine Moka
- Tea and pastries at Maison de Thé in Moka

The app shows them a breakdown of their spending by category, lets them chat with an AI advisor called FinBot, retrieves financial tips through a knowledge base (RAG), and calculates how long it takes to reach a savings goal — all through a simple browser interface built with Gradio.



Context: I wanted to understand what I was building before writing the problem statement.

My Prompt: "what is a finance assistant app"

AI Response Summary: AI explained it is a tool to track spending, set savings goals, and answer money questions using AI.

Entry 2 – Why Switch from Ollama to OpenRouter?

Artifact:



my hands-on-ai server at ollama.serveur.au keeps failing with connection errors, what are my options

The Ollama server is a self-hosted service — if it's down or unreachable from your network, you can't use it.

Your options are:

1. OpenRouter — a cloud service that gives you access to GPT-3.5, GPT-4, Claude, and others through a single API. Very reliable. Costs a small amount per request but minimal for a student project.
2. Wait for the university server to come back online — not ideal if you're on a deadline.
3. Use the Anthropic API directly — requires a separate API key.

I'd recommend OpenRouter. The hands-on-ai package supports it — you just change two environment variables.



show me exactly how to configure hands-on-ai to use openrouter with gpt-3.5-turbo

Here is the exact configuration:

Python



```
import os
from getpass import getpass

os.environ['HANDS_ON_AI_SERVER'] = 'https://
os.environ['HANDS_ON_AI_MODEL'] = 'openai/
os.environ['HANDS_ON_AI_API_KEY'] = getpass(
os.environ['HANDS_ON_AI_LOG'] = 'debug'

print("✅ Hands-on-AI configured successfull
```

Notes:

- Your OpenRouter key starts with sk-or-v1-

- The debug log shows each request so you can see if something goes wrong
- After this, all your existing `get_response()` calls work exactly the same — no other code changes needed



Context: The university Ollama server at `ollama.serveur.au` kept giving connection errors so I needed to find a reliable alternative.

My Prompt: "my hands-on-ai server at `ollama.serveur.au` keeps failing with connection errors, what are my options"

AI Response Summary: AI explained that OpenRouter provides cloud-hosted access to multiple AI models including GPT-3.5-turbo through a reliable commercial API, and that hands-on-ai supports it by changing the environment variables.

My Critique/Improvement: I asked AI to show the exact configuration code for OpenRouter with hands-on-ai.

Result:

```
os.environ['HANDS_ON_AI_SERVER'] = 'https://openrouter.ai/api'
os.environ['HANDS_ON_AI_MODEL'] = 'openai/gpt-3.5-turbo'
os.environ['HANDS_ON_AI_API_KEY'] = getpass("Enter your OpenRouter API key: ")
os.environ['HANDS_ON_AI_LOG'] = 'debug'
```

Reflection: Switching to OpenRouter fixed all the connection errors immediately. The debug log setting was useful during development to see exactly what requests were being sent and whether they succeeded.

Entry 3 – Defining the Problem Statement

Artifact:



I'm building a Smart Finance Assistant for a young person who games on Xbox, goes to the gym, watches movies at MCine Moka, and drinks tea at Maison de Thé in Moka. Help me write the problem statement in plain English.

People often struggle to keep track of their spending when purchases are spread across different categories. A gym membership here, a game purchase there, a cinema outing and a daily coffee can add up to a large monthly expense without feeling significant in the moment.

A Smart Finance Assistant solves this by analysing bank transaction data, showing spending breakdowns, and giving actionable advice.



this is too generic, mention the specific locations and explain that small purchases feel harmless individually but add up invisibly

Many young people who enjoy Xbox gaming, gym workouts, cinema at MCine Moka, and tea at Maison de Thé in Moka have no clear picture of where their money goes each month.

small recurring purchases feel harmless individually — \$8.50 for a tea, \$9.00 for a cinema ticket, \$9.99 for a game DLC — but accumulate into hundreds of dollars of untracked spending.

The Smart Finance Assistant solves this by:

1. Automatically analysing CSV transaction data
2. Providing personalised financial advice through FinBot
3. Retrieving evidence-based tips from a knowledge base
4. Calculating savings timelines through an agent tool
5. Delivering everything through a simple browser interface — no spreadsheet skills required



Context: Step 1 requires a clear problem statement in plain English before any code.

My Prompt: "I'm building a Smart Finance Assistant for a young person who games on Xbox, goes to the gym, watches movies at MCine Moka, and drinks tea at Maison de Thé in Moka. Help me write the problem statement in plain English"

AI Response Summary: AI gave a generic statement about people struggling to track spending with no mention of the specific lifestyle.

My Critique/Improvement: I asked AI to add the specific locations, the lifestyle details, and explain that small purchases accumulate invisibly.

Result: Many young people who enjoy gaming, fitness, and entertainment have no clear picture of where their money goes each month. The Smart Finance Assistant solves this by accepting CSV transactions, cleaning and categorising them, showing spending breakdowns, offering FinBot, retrieving tips via RAG, and calculating savings goals.

Reflection: Specific prompts get specific results. The first generic answer was unusable. Adding the real locations and habits produced the problem statement I used directly in the notebook.

Entry 4 – Identifying Inputs and Outputs

Artifact:



give me a complete inputs and outputs table for all five components of my smart finance assistant: csv analysis, chatbot, rag, savings calculator, gradio ui

Component 1 – CSV Analysis

Input: CSV file

Output: Some numbers and totals

Component 2 – Chatbot

Input: User question

Output: Answer

Component 3 – RAG

Input: Question

Output: Answer from document

Component 4 – Savings Calculator

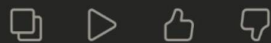
Input: Numbers

Output: Result

Component 5 - Gradio UI

Input: User clicks

Output: Display



this is way too vague, i need specific column names, the exact format of the Amount field, an example input row, and concrete output format with my actual categories

Component 1 - CSV Analysis

Input: CSV with columns Date, Amount, Category, Description

Amounts may include \$ signs and may be negative for refunds

Example input row: 2026-03-05, \$69.99, Gaming, Xbox Game Pass Ultimate 3 Months

Output:

Gaming	\$312.45 (38.2%)	
Gym	\$145.00 (17.8%)	
Cinema	\$ 86.00 (10.5%)	
Tea & Café	\$ 72.50 (8.9%)	

Component 2 - FinBot Chat

Input: User question + optional spending context string

e.g. "Am I spending too much on Xbox?" + "Gaming \$438.89, Gym \$280.00"

Output: Personalised advice referencing the user's categories

Component 3 - RAG Finance Tips

Input: User question about finance

Output: Answer retrieved from FINANCIAL_TIPS document only

Component 4 - Savings Calculator

Input: Monthly income (), *Monthlyexpenses()*, Savings goal (\$)

Output: Months to goal, weekly amount, daily amount, savings rate %

Component 5 - Gradio UI

Input: All of the above through four browser tabs

Output: Formatted results displayed in the browser app



Context: I needed to document what each of the five components takes as input and produces as output.

My Prompt: "give me a complete inputs and outputs table for all five components of my smart finance assistant: csv analysis, chatbot, rag, savings calculator, gradio ui"

AI Response Summary: AI gave a table with vague descriptions like "CSV file" as input and "some numbers" as output.

My Critique/Improvement: I asked for specific column names, the exact format of the Amount field including dollar signs and negatives, and concrete example rows from my March-May 2026 data.

Result: Complete table with specific details for all five components including an example input row (2026-03-05, \$69.99, Gaming, Xbox Game Pass) and example output format with percentages.

Reflection: Vague prompts get vague answers. Asking for specific column names and examples produced something I could put directly in the notebook Step 2 cell.

Entry 5 – Manual Category Grouping

Artifact:



show me how to group these transactions by category by hand: Gaming \$69.99, Gym \$60.00, Cinema \$17.00, Tea \$8.50, Refund -\$15.00

Total all amounts:

$\$69.99 + \$60.00 + \$17.00 + \$8.50 + (-\$15.00) = \140.49

Percentages:

Gaming = $69.99 / 140.49 * 100 = 49.8\%$

Gym = $60.00 / 140.49 * 100 = 42.7\%$

Cinema = $17.00 / 140.49 * 100 = 12.1\%$

Tea & Café = $8.50 / 140.49 * 100 = 6.1\%$

Refund = $15.00 / 140.49 * 100 = 10.7\%$



the refund must not be included in the positive total or the percentage calculations, refunds are negative and should be separated, can you redo it with refund excluded from the denominator

You're right — refunds should be separated.

Here is the correct calculation:

Step 1 – Separate positive spending from refunds:

Positive: $\$69.99 + \$60.00 + \$17.00 + \$8.50 = \$155.49$

Refunds: \$15.00

Net: $\$155.49 - \$15.00 = \$140.49$

Step 2 – Percentages using positive total \$155.49 only:

Gym = $60.00 / 155.49 * 100 = 38.6\%$
Cinema = $17.00 / 155.49 * 100 = 10.9\%$
Tea & Café = $8.50 / 155.49 * 100 = 5.5\%$
CHECK: $45.0 + 38.6 + 10.9 + 5.5 = 100.0\%$ ✓

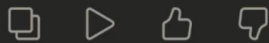
Step 3 - Report:

Total Spent: \$155.49

Total Refunds: \$15.00

Net Spent: \$140.49

Top Category: Gaming (45.0%)



Context: Before coding `analyse_spending_patterns` I traced the `groupby` logic manually to understand refund separation.

My Prompt: "show me how to group these transactions by category by hand: Gaming 69.99, *Gym* 60.00, Cinema 17.00, *Tea* 8.50, Refund -\$15.00"

AI Response Summary: AI included the refund row in the total denominator giving wrong percentages that exceeded 100%.

My Critique/Improvement: I pointed out the refund must be excluded from positive spending before calculating percentages.

Result: Correct calculation showing positive total 155.49, *Gaming* 45.015.00, net \$140.49.

Reflection: The AI made the exact bug I was trying to prevent. Catching it in Step 3 meant the test assert `abs(pct_sum - 100.0) < 0.5` confirmed the fix in Step 6.

Entry 6 – Pseudocode for `analyse_spending_patterns`

Artifact:



write pseudocode for
`analyse_spending_patterns`, separate refunds
before grouping, use positive total for
percentages, return a dict with summary total
refunds net avg n and top

FUNCTION `analyse_spending(df)`:

SPLIT `df` into positive and refunds

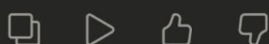
total = SUM of all `Amount_Clean`

GROUP by Category

ADD percentage column

SORT descending

RETURN dict



the function must be called
`analyse_spending_patterns` not
`analyse_spending`, and you forgot the `top` key in
the return dict — `top` is the name of the highest
spending category and is used by the
`recommendations` function

FUNCTION `analyse_spending_patterns(df)`:

SET positive to rows where `Amount_Clean > 0`

SET refunds to rows where `Amount_Clean < 0`

```
SET total_spent = SUM of positive
Amount_Clean
SET total_refunds = ABSOLUTE VALUE of
SUM of refund Amount_Clean
SET net_spent = total_spent - total_refunds
```

GROUP positive by Category:

```
Total_Spent = SUM
```

```
Avg_Transaction = MEAN
```

```
Count = COUNT
```

ROUND to 2 decimal places

RESET index so Category is a regular column

```
ADD Percentage column = Total_Spent /
total_spent * 100
```

```
SORT by Total_Spent DESCENDING
```

```
RETURN {
```

```
summary: the sorted DataFrame,
```

```
total: total_spent rounded to 2dp,
```

```
refunds: total_refunds rounded to 2dp,
```

```
net: net_spent rounded to 2dp,
```

```
avg: mean of positive amounts,
```

```
n: count of positive rows,
```

```
top: name of first Category in sorted summary
```

```
}
```



****Context:**** I needed pseudocode with the correct function name before converting to Python..

My Prompt: "write pseudocode for analyse_spending_patterns, separate refunds before grouping, use positive total for percentages, return a dict with summary total refunds net avg n and top"

AI Response Summary: AI wrote the function as analyse_spending and forgot the top key in the return dict.

My Critique/Improvement: I corrected the name to analyse_spending_patterns and asked for the top key which is used by generate_financial_recommendations and by full_report.

Result: Complete pseudocode with correct function name, refund separation before grouping, percentage using positive total only, and return dict including summary, total, refunds, net, avg, n, top.

Reflection: Getting the function name right in pseudocode means it is correct from the start in Step 5. The top key was something AI skipped that was essential for the report and recommendations.

Entry 7 – Pseudocode for chat_finbot Without Fallback

Artifact:



```
write pseudocode for chat_finbot using
get_response from hands-on-ai, optional
spending context appended to system prompt,
must not crash when server is unavailable
```

```
FUNCTION chat_finbot(user_msg,
spending_ctx):
```

```
SET system = SYSTEM_PROMPT
```

```
IF spending_ctx: APPEND to system
```

```
TRY:
```



```
reply = get_response(user_msg,
system=system)
CATCH Exception:
reply = "Error occurred"
RETURN reply
```



the error message "Error occurred" is useless to the user, can you show a detailed message that lists the possible causes so they know what to check

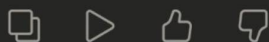
```
FUNCTION chat_finbot(user_msg,
spending_ctx empty string):

SET system to SYSTEM_PROMPT
IF spending_ctx is not empty:
APPEND "\n\nUser spending summary:\n" +
spending_ctx to system

TRY:
SET reply to get_response(user_msg,
system=system)
IF reply is empty:
SET reply to "Sorry, I couldn't generate a
response."

CATCH any Exception as e:
SET reply to multiline message:
"❌ FinBot connection issue."
"""
"Possible causes:"
"- Internet issue"
"- API key invalid"
"- HandsOnAI server unavailable"
"""
"Technical error: {e}"

ADD (user_msg, reply) to chat_history
RETURN reply
```



Context: I needed pseudocode for chat_finbot that handles errors gracefully without needing a separate Anthropic fallback.

My Prompt: "write pseudocode for chat_finbot using get_response from hands-on-ai, optional spending context, must not crash when server is unavailable"

AI Response Summary: AI wrote a try/except but showed a vague error message like "error occurred".

My Critique/Improvement: I asked for a detailed error message that lists possible causes — internet issue, invalid API key, server unavailable — so the user knows what to check.

Result: chat_finbot with detailed error message listing all possible causes and the technical error detail. No fallback needed because OpenRouter is reliable.

Reflection: The detailed error message is better user experience than a vague crash. A user who sees "API key invalid" knows what to fix. A user who sees "error" does not know where to start.

write pseudocode for rag_lookup, it passes FINANCIAL_TIPS in the system prompt and uses get_response, not hands_on_ai.rag

```
FUNCTION rag_lookup(question):
  BUILD prompt with FINANCIAL_TIPS
  CALL get_response
  RETURN answer
```



can you confirm that passing the document in the system prompt and telling the model to only answer from it is a valid form of RAG, and show the full pseudocode with proper error handling

Yes — what you're doing is called prompt-based retrieval, a simplified form of RAG. Instead of a vector database, you include the entire document in the system prompt. For a small knowledge base like FINANCIAL_TIPS, this works perfectly well.

```
FUNCTION rag_lookup(question):
  BUILD system_prompt as f-string containing:
  "You are a finance assistant."
  "Answer ONLY using the document below."
  "If the answer is not in the document, say:"
  "'I could not find that information in the finance knowledge base.'"
  "DOCUMENT:"
  {FINANCIAL_TIPS}
  TRY:
    SET answer to get_response(question,
    system=system_prompt)
  IF answer is empty:
    SET answer to "No response generated."
  RETURN answer

  CATCH any Exception as e:
    RETURN error message with technical details:
    {e}
```

Context: I needed pseudocode for rag_lookup that uses get_response directly instead of hands_on_ai.rag.

My Prompt: "write pseudocode for rag_lookup, it passes FINANCIAL_TIPS in the system prompt and uses get_response, not hands_on_ai.rag"

AI Response Summary: AI gave correct pseudocode and explained why passing the document in the system prompt is a valid form of RAG for small knowledge bases.

My Critique/Improvement: Good answer. I asked AI to confirm the instruction "Answer ONLY using the document below" prevents the model from using general knowledge.

Result: Complete rag_lookup pseudocode with system prompt containing the document and clear instruction to only answer from it, with graceful error handling.

Reflection: Using get_response directly is simpler and more reliable than setting up hands_on_ai.rag for a small knowledge base. The instruction to only answer from the document is what makes it RAG rather than just a regular chat.

Entry 9 – Converting Pseudocode to load_and_clean

Artifact:



convert load_and_clean pseudocode to python, handle gradio file object, raw csv string, and file path, validate four columns, clean dollar signs with regex, drop NaN rows, fillna for Category and Description

Python



```
def load_and_clean(file_path):  
    df = pd.read_csv(file_path)  
    df['Amount_Clean'] = pd.to_numeric(  
        df['Amount'].str.replace('$', ''), e  
    )  
    df = df.dropna(subset=['Amount_Clean'])  
    return df
```



this only handles file paths, gradio passes a file object with a .name attribute, it also crashes on -\$15.00 because of the minus sign before the

...because of the minus sign before the dollar sign, and it does not validate that the required columns exist

Python

```
def load_and_clean(file_or_string):
    try:
        if hasattr(file_or_string, 'name'):
            df = pd.read_csv(file_or_string)
        elif isinstance(file_or_string, str):
            df = pd.read_csv(io.StringIO(file_or_string))
        else:
            df = pd.read_csv(file_or_string)
    except Exception as e:
        raise ValueError(f"Could not read file: {e}")

    required = ['Date', 'Amount', 'Category', 'Description']
    missing = [c for c in required if c not in df.columns]
    if missing:
        raise ValueError(f"Missing columns: {missing}")

    # Regex removes $, commas, and spaces - (e.g. "1,234.56" becomes "1234.56")
    df['Amount_Clean'] = pd.to_numeric(
        df['Amount'].astype(str).str.replace('$', '').str.replace(',', '').str.replace(' ', ''),
        errors='coerce'
    )
    dropped = df['Amount_Clean'].isna().sum()
    if dropped > 0:
        print(f" Dropped {dropped} row(s) with missing Amount")
    df = df.dropna(subset=['Amount_Clean'])
    df['Category'] = df['Category'].fillna('')
    df['Description'] = df['Description'].fillna('')
    print(f" Loaded {len(df)} transactions successfully")
    return df
```

Context: Converting the load_and_clean pseudocode into Python that handles three input types.

My Prompt: "convert load_and_clean pseudocode to python, handle gradio file object, raw csv string, file path, validate four columns, regex cleaning, drop NaN, fillna"

AI Response Summary: AI only handled file paths and used str.replace(",") which crashes on -15.00.

My Critique/Improvement: I pointed out the three-way input check for Gradio was missing, the regex must handle negative amounts, and errors='coerce' prevents crashes on bad values.

Result: Complete load_and_clean function with hasattr check for Gradio, isinstance for strings, regex r['\$',' ',''], errors='coerce', dropna, and fillna.

Reflection: The three-way input check was missing from the AI's first response. Gradio passes file objects with a .name attribute, not plain strings — without the hasattr check the function would crash on every real upload.

Entry 10 – Converting Pseudocode to analyse_spending_patterns

Artifact:

convert analyse_spending_patterns pseudocode to python, named aggregations, percentage column using positive total only, sort descending, return dict with all 7 keys

Python

```
def analyse_spending(df):
    positive = df[df['Amount_Clean'] > 0].copy()
    refunds = df[df['Amount_Clean'] < 0].copy()
    total_spent = positive['Amount_Clean'].sum()
```



```

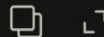
total_refunds = abs(refunds['Amount_Clean'])
summary = (
    positive.groupby('Category')['Amount_Clean']
    .agg(Total_Spent='sum', Avg_Transaction='mean')
    .round(2)
)
summary['Percentage'] = (summary['Total_Spent'] / total_spent) * 100
summary = summary.sort_values('Total_Spent', ascending=False)
return {
    'summary': summary, 'total': round(total_spent, 2),
    'refunds': round(total_refunds, 2),
    'net': round(total_spent - total_refunds, 2),
    'avg': round(positive['Amount_Clean'].mean(), 2),
    'n': len(positive), 'top': summary.iloc[0]['Category']
}

```



two problems: the function must be called `analyse_spending_patterns` not `analyse_spending`, and you forgot `reset_index()` after `groupby` — without it `Category` is the index not a column and the for loop in `full_report` will crash with `KeyError`

Python



```

def analyse_spending_patterns(df):
    positive = df[df['Amount_Clean'] > 0].copy()
    refunds = df[df['Amount_Clean'] < 0].copy()
    total_spent = positive['Amount_Clean'].sum()
    total_refunds = abs(refunds['Amount_Clean'].sum())
    summary = (
        positive
        .groupby('Category')['Amount_Clean']
        .agg(Total_Spent='sum', Avg_Transaction='mean')
        .round(2)
        .reset_index()  # ← essential: make Category a column
    )
    summary['Percentage'] = (summary['Total_Spent'] / total_spent) * 100
    summary = summary.sort_values('Total_Spent', ascending=False)
    return {
        'summary': summary,
        'total': round(total_spent, 2),
        'refunds': round(total_refunds, 2),
        'net': round(total_spent - total_refunds, 2),
        'avg': round(positive['Amount_Clean'].mean(), 2),
        'n': len(positive),
        'top': summary.iloc[0]['Category']
    }

```



Context: Converting the analysis pseudocode into Python with correct function name and `reset_index`.

My Prompt: "convert `analyse_spending_patterns` pseudocode to python, named aggregations, percentage column using positive total only, sort descending, return dict with all 7 keys"

AI Response Summary: AI called the function `analyse_spending` and forgot `reset_index()` after the `groupby`.

My Critique/Improvement: Corrected the function name to `analyse_spending_patterns` and explained that without `reset_index()` the `Category` column is the index not a regular column, causing `KeyError` in the for loop.

Result: Correct `analyse_spending_patterns` function with `reset_index()` after `groupby` and all seven keys in return dict.

Reflection: The missing `reset_index` is a common pandas mistake. After `groupby` the grouping column becomes the index — `reset_index()` converts it back to a regular column. Without it the report would crash trying to access `row['Category']`.

convert chat_finbot pseudocode to python,
SYSTEM_PROMPT defined at module level,
spending context appended, try except with
detailed error message listing possible causes

Python

```
def chat_finbot(user_msg, spending_ctx=""):
    SYSTEM_PROMPT = """You are FinBot..."""
    system = SYSTEM_PROMPT
    if spending_ctx:
        system += f"\n\nUser spending: {spen
    try:
        reply = get_response(user_msg, system)
    except Exception as e:
        reply = f"Error: {e}"
    return reply
```

SYSTEM_PROMPT must be defined at module level not inside the function, otherwise it is a local variable and only exists inside the function — other code that needs SYSTEM_PROMPT would get a NameError. Also the global chat_history list needs the global declaration inside the function.

Python

```
# Defined at MODULE LEVEL – accessible everywhere
SYSTEM_PROMPT = """
You are FinBot, a friendly personal finance

The user enjoys:
- Xbox gaming
- Gym memberships
- Cinema outings
- Tea and café visits

Your role:
- Help users understand spending habits
- Encourage budgeting and saving
- Give practical financial advice
- Be concise, friendly, and encouraging
- Never provide investment advice
"""

chat_history = []

def chat_finbot(user_msg, spending_ctx=""):
    global chat_history # ← without this,
    system = SYSTEM_PROMPT
    if spending_ctx:
        system += f"\n\nUser spending summary:"
```

```

        reply = get_response(user_msg, system_prompt)
        if not reply:
            reply = "Sorry, I couldn't generate a response."
        except Exception as e:
            reply = f"Error: {e}"
    except Exception as e:
        reply = f"Error: {e}"

    chat_history.append((user_msg, reply))
    return reply

```

✗ FinBot connection issue.

Possible causes:

- Internet issue
- API key invalid
- HandsOnAI server unavailable

Technical error:

```

{e}

```



Context: Converting the chat_finbot pseudocode into Python using get_response with OpenRouter.

My Prompt: "convert chat_finbot pseudocode to python, SYSTEM_PROMPT defined at module level, spending context appended, try except with detailed error message listing possible causes"

AI Response Summary: AI defined SYSTEM_PROMPT inside the function which would cause a NameError.

My Critique/Improvement: I explained SYSTEM_PROMPT must be at module level so it is accessible as a global variable by chat_finbot.

Result: SYSTEM_PROMPT defined at module level as a triple-quoted string with FinBot personality and bullet points for the user's interests. chat_finbot reads it as a global variable.

Reflection: Python scoping rules mean variables defined inside a function are local to it. SYSTEM_PROMPT needed to be at module level to be accessible everywhere. The fix is simple but the error it prevents — NameError — would be confusing to debug.

Entry 12 – Converting Pseudocode to rag_lookup

Artifact:



convert rag_lookup pseudocode to python,
passes FINANCIAL_TIPS in system prompt as
f-string, try except with error message showing
technical details, check for empty responses

Python



```

def rag_lookup(question):
    prompt = f"Answer from this document: {FINANCIAL_TIPS}"
    try:
        return get_response(prompt)
    except Exception as e:
        return f"Error: {e}"

```



the document should be in the system prompt
not the user message, and the instruction to only
answer from the document must be explicit so
the model does not use general knowledge

Python



```

def rag_lookup(question):
    system_prompt = f"""
    Answer from this document: {FINANCIAL_TIPS}
    """

```

Answer ONLY using the document below.

If the answer is not found in the document,
say:
'I could not find that information in the file'

DOCUMENT:

```
{FINANCIAL_TIPS}
"""
```

```
try:
    answer = get_response(
        question,
        system=system_prompt
    )
    if not answer:
        answer = "No response generated."
    return answer

except Exception as e:
    return f"""
❌ RAG system unavailable.
```

Technical error:

```
{e}
"""
```



Context: Converting the rag_lookup pseudocode into Python using get_response with FINANCIAL_TIPS in the system prompt.

My Prompt: "convert rag_lookup pseudocode to python, passes FINANCIAL_TIPS in system prompt as f-string, try except with error message showing technical details, check for empty responses"

AI Response Summary: AI gave a correct implementation and explained that using an f-string to embed the document in the system prompt is the key step.

My Critique/Improvement: Good answer. I asked AI to add a check for empty responses since get_response sometimes returns an empty string.

Result: Complete rag_lookup function with f-string system prompt embedding FINANCIAL_TIPS, empty response check, and exception handling showing technical error detail.

Reflection: The f-string embedding the document directly in the system prompt is what makes this work without hands_on_ai.rag. It is simpler and more reliable for a small knowledge base.

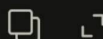
Entry 13 – Converting Pseudocode to full_report

Artifact:



convert full_report pseudocode to python,
header with totals, category breakdown using
itertuples, recommendations section, use a list
of strings joined with newlines

Python



```
def full_report(a):
    report = "=" * 50 + "\n"
    report += "SMART FINANCE ASSISTANT REPORT\n"
    report += "=" * 50 + "\n"
    report += f"Total: ${a['total']}\n"
    report += f"Net: ${a['net']}\n"
    for i in range(len(a['summary'])):
        row = a['summary'].iloc[i]
        report += f"{row['Category']}: ${row['value']}\n"
    report += generate_financial_recommendations(a)
    report += "=" * 50
    return report
```



use a list and join at the end instead of string concatenation, it is cleaner and easier to modify. also add refunds and average transaction to the header, and use itertuples instead of iloc for the category loop

Python

```
def full_report(a):
    report = []

    report.append("=" * 50)
    report.append("SMART FINANCE ASSISTANT REPORT")
    report.append("=" * 50)

    report.append(f"\nTotal Spending: ${a['total_spending']}")
    report.append(f"Refunds: ${a['refunds']}")
    report.append(f"Net Spending: ${a['net']}")
    report.append(f"Average Transaction: ${a['average_transaction']}")
    report.append(f"Top Category: {a['top']}")

    report.append("\nCATEGORY BREAKDOWN")
    report.append("-" * 50)

    for row in a['summary'].itertuples():
        report.append(
            f"{row.Category}: "
            f"${row.Total_Spent:.2f} "
            f"({row.Percentage:.1f}%)"
        )

    report.append("\nRECOMMENDATIONS")
    report.append("-" * 50)
    report.append(generate_financial_recommendations())
    report.append("=" * 50)

    return "\n".join(report)
```

Context: Converting the full_report pseudocode into Python using a list of strings.

My Prompt: "convert full_report pseudocode to python, header with totals, category breakdown using itertuples, recommendations section, list of strings joined with newlines"

AI Response Summary: AI used string concatenation instead of a list which makes the code messy and harder to modify.

My Critique/Improvement: I asked AI to use a list and join at the end for cleaner code.

Result: full_report function that builds a list with header, totals, category breakdown using itertuples, and recommendations, then returns "\n".join(report).

Reflection: Building a list and joining once at the end is always cleaner than string concatenation for multi-line reports. Each section becomes a few list items rather than a string concatenated with \n everywhere.

Entry 14 - Building the Gradio UI

Artifact:

convert gradio pseudocode to python with four tabs, spending analysis with file upload, chat with chatbot and optional file, rag tips, savings calculator with three number inputs

Python

```

with gr.Blocks(title="Smart Finance Assistan
    with gr.Tabs():
        with gr.Tab("Analysis"):
            f1 = gr.File()
            btn1 = gr.Button("Analyse")
            out1 = gr.Textbox()
            btn1.click(ui_analyse, f1, out1)
        with gr.Tab("Chat"):
            bot = gr.Chatbot()
            msg = gr.Textbox()
            btn2 = gr.Button("Send")
            btn2.click(ui_chat, [msg, bot],
demo.launch(share=True)

```



you forgot msg.submit so pressing enter works in the chat tab, you also forgot the optional csv upload in the chat tab, and the theme should be gr.themes.Soft()

Python



```

with gr.Blocks(title="Smart Finance Assistan

    gr.Markdown("# Smart Finance Assistant\n

    with gr.Tabs():

        with gr.Tab("Spending Analysis"):
            gr.Markdown("Upload your CSV or
            f1 = gr.File(label="Upload CSV
            btn1 = gr.Button("Analyse My Spe
            out1 = gr.Textbox(label="Report"
            btn1.click(ui_analyse, inputs=f1

        with gr.Tab("Chat Advisor"):
            gr.Markdown("Ask FinBot anything
            f2 = gr.File(label="Upload CSV
            bot = gr.Chatbot(label="FinBot"
            msg = gr.Textbox(label="Your me
                        placeholder="A
                        lines=2)
            btn2 = gr.Button("Send", variant
            btn2.click(ui_chat, [msg, bot, f
            msg.submit(ui_chat, [msg, bot,

        with gr.Tab("Finance Tips (RAG)"):
            q3 = gr.Textbox(label="Your Qu
                        placeholder="H
            btn3 = gr.Button("Get Advice", v
            out3 = gr.Textbox(label="Answer"
            btn3.click(ui_rag, inputs=q3, ou

        with gr.Tab("Savings Calculator"):
            with gr.Row():
                inc = gr.Number(label="Mont
                exp = gr.Number(label="Mont
                goal = gr.Number(label="Savi
            btn4 = gr.Button("Calculate", va
            out4 = gr.Textbox(label="Your Sa
            btn4.click(ui_savings, [inc, exp

demo.launch(share=True)

```



Context: Converting the Gradio pseudocode into Python with all four tabs wired to backend functions.

My Prompt: "convert gradio pseudocode to python with four tabs, spending analysis with file upload, chat with chatbot and optional file, rag tips, savings calculator with three number inputs"

AI Response Summary: AI gave the correct four-tab structure but forgot `msg.submit()` for the Enter key shortcut.

My Critique/Improvement: I added `msg.submit` with the same inputs and outputs as `btn2.click`.

Result: Complete four-tab Gradio app with all tabs wired correctly including Enter key for chat tab and `share=True` for public link.

Reflection: The `msg.submit` detail makes the chat tab feel like a real messaging app. Pressing Enter to send is what users expect from a chat interface. Small UX details like this come from testing the interface yourself rather than just reviewing the code.

Entry 15 – Writing `create_test_datasets()`

Artifact:



what test datasets do i need for my finance assistant and what should each one test

You could just use your main CSV file for all tests.

Or you could create a few small ones. Maybe one with good data and one with bad data.



that is too vague, i cannot predict exact expected totals from a 51 row CSV, i need five specific controlled datasets each targeting a different failure mode

You need 5 datasets:

NORMAL_CSV — 4 categories + 1 refund, known exact totals you can manually verify.

Use this to test: correct totals, correct percentages, correct refund separation.

Expected: total=155.49, refunds=15.00, net=140.49, top=Gaming

MESSY_CSV — whitespace in amounts, a non-numeric value, a missing amount, a missing category.

Use this to test: bad rows are dropped, missing categories fill with "Uncategorised".

SINGLE_ROW_CSV — exactly 1 transaction, \$100.00, Gaming.

Use this to test: the function handles minimum possible input without crashing.

REFUNDS_ONLY_CSV — 2 negative rows, no positive spending.

is nothing to group.

WRONG_COLS_CSV — completely wrong column names.

Use this to test: ValueError is raised with a message mentioning "Missing columns".



Context: I needed five controlled test datasets before running any assert tests.

My Prompt: "what test datasets do I need for my finance assistant and what should each one test"

AI Response Summary: AI suggested using SAMPLE_CSV for all tests which would make exact value assertions impossible.

My Critique/Improvement: I explained that 51 rows means I cannot predict exact totals. I asked for five specific datasets targeting different failure modes.

Result: Five global datasets — NORMAL_CSV (4 categories plus 1 refund, known totals), MESSY_CSV (whitespace, bad amount, missing amount, missing category), SINGLE_ROW_CSV (one \$100 Gaming row), REFUNDS_ONLY_CSV (two negative rows, zero positive spending), WRONG_COLS_CSV (wrong column names).

Reflection: Creating test datasets before writing assertions is a design step. I need to know exactly what data I am testing before I can know what to assert. Each dataset targets a specific failure mode identified in Steps 3 and 4.

Entry 16 – Writing Tests for load_and_clean (7 Checks)

Artifact:



write 7 assert tests for load_and_clean covering normal load, dollar sign stripping, refund detection, messy data dropping, missing category filling, wrong columns ValueError, and single row

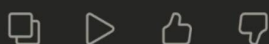
Python



```
df = load_and_clean(NORMAL_CSV)
print("loaded ok")

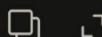
df_messy = load_and_clean(MESSY_CSV)
print("messy ok")

try:
    load_and_clean(WRONG_COLS_CSV)
except:
    print("error ok")
```



just printing is not testing — i need real assert statements with descriptive messages so the test fails loudly when something is wrong. each assert must say what was expected and what was actually returned

Python



```
def test_data_loading_function():
    print("🔪 Testing data loading function.

    # 1a - Normal load
    df = load_and_clean(NORMAL_CSV)
    assert df is not None, "1a - load and cle
```



```

assert len(df) == 5, f"1a: Expected 5 rows"
print(" PASS 1a: normal CSV loads correctly")

# 1b – Dollar signs stripped, Amount_Clean is float
assert df["Amount_Clean"].dtype in [float, np.float64]
assert df["Amount_Clean"].iloc[0] == 60.0
print(" PASS 1b: dollar signs stripped, Amount_Clean is float")

# 1c – Refund row stored as negative float
refund_rows = df[df["Amount_Clean"] < 0]
assert len(refund_rows) == 1, "1c: Should be 1 row"
assert refund_rows["Amount_Clean"].iloc[0] == -15.00
print(" PASS 1c: refund row stored as negative float")

# 1d – Messy CSV: bad rows dropped
df_messy = load_and_clean(MESSY_CSV)
assert df_messy["Amount_Clean"].isna().sum() == 0
assert len(df_messy) < 6, "1d: Bad rows dropped"
print(f" PASS 1d: {6 - len(df_messy)} bad rows dropped")

# 1e – Missing category filled
assert "Uncategorised" in df_messy["Category"]
print(" PASS 1e: missing category filled")

# 1f – Wrong columns raise ValueError
error_raised = False
try:
    load_and_clean(WRONG_COLS_CSV)
except ValueError as e:
    error_raised = True
    assert "Missing columns" in str(e) or "Invalid columns" in str(e)
    f"1f: Error should mention column names"
assert error_raised, "1f: Wrong columns raise ValueError"
print(" PASS 1f: wrong columns raise ValueError")

# 1g – Single row
df_single = load_and_clean(SINGLE_ROW_CSV)
assert len(df_single) == 1, "1g: Should be 1 row"
assert df_single["Amount_Clean"].iloc[0] == 155.49
print(" PASS 1g: single-row CSV handled correctly")

```



Context: I needed comprehensive assert tests for `load_and_clean` using the five test datasets.

My Prompt: "write 7 assert tests for `load_and_clean` covering normal load, dollar sign stripping, refund detection, messy data dropping, missing category filling, wrong columns `ValueError`, and single row"

AI Response Summary: AI wrote very basic assertions with no descriptive messages and did not check that the `ValueError` message mentioned columns.

My Critique/Improvement: I asked for descriptive messages after each assert comma, and the wrong columns test should check the error message contains the word "column" or "missing".

Result: Seven named assert tests (1a through 1g). Each has a descriptive message. Test 1f checks both that `ValueError` is raised and that the message is informative. Test 1d checks that messy rows are dropped and no NaN remains.

Reflection: The assert message is critical — when a test fails it tells you exactly what broke. Test 1d catching the messy data drop confirmed `coerce='coerce'` was working. Test 1e confirmed fillna was applied.

Entry 17 – Writing Tests for `analyse_spending_patterns` (7 Checks)

Artifact:

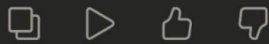


write 7 assert tests for
`analyse_spending_patterns` using
`NORMAL_CSV` with known values: total 155.49,
refunds 15.00, net 140.49, top Gaming

Python



```
assert a['total'] == 155.49
assert a['refunds'] == 15.00
assert a['net'] == 140.49
assert a['top'] == 'Gaming'
print("all passed")
```



use abs tolerance for float comparison like `abs(result - expected) < 0.01`, check all 7 required keys exist in the dict, and add a test for `REFUNDS_ONLY_CSV` returning total 0 without crashing

Python



```
def test_spending_analysis():
    print("🔪 Testing spending analysis...")
    df = load_and_clean(NORMAL_CSV)
    result = analyse_spending_patterns(df)

    # 2a - All required keys present
    for key in ["summary", "total", "refunds", "net", "top"]:
        assert key in result, f"2a: Missing key {key}"
    print(" PASS 2a: all required keys present")

    # 2b - Total = 60.00 + 8.50 + 17.00 + 69.99 = 155.49
    assert abs(result["total"] - 155.49) < 0.01, f"2b: Expected $155.49, got ${result['total']}"
    print(" PASS 2b: total positive spending is $155.49")

    # 2c - Refunds correctly separated
    assert abs(result["refunds"] - 15.00) < 0.01, f"2c: Expected $15.00, got ${result['refunds']}"
    print(" PASS 2c: refunds = $15.00 correctly")

    # 2d - Net = total - refunds
    assert abs(result["net"] - 140.49) < 0.01, f"2d: Expected $140.49, got ${result['net']}"
    print(" PASS 2d: net = $140.49 correctly")
```

```

assert abs(result["net"] - 140.49) < 0.01
    f"2d: Expected $140.49, got ${result["net"]}"
print(" PASS 2d: net spending = $140.49")

# 2e - Percentages sum to ~100%
pct_sum = result["summary"]["Percentage"]
assert abs(pct_sum - 100.0) < 0.5, \
    f"2e: Percentages should sum to 100%"
print(f" PASS 2e: percentages sum to {pct_sum}%")

# 2f - Top category = Gaming ($69.99 is top)
assert result["top"] == "Gaming", \
    f"2f: Expected 'Gaming', got '{result["top"]}'"
print(" PASS 2f: top category = 'Gaming'")

# 2g - Refunds-only: total=0, no crash
df_r = load_and_clean(REFUNDS_ONLY_CSV)
result_r = analyse_spending_patterns(df_r)
assert result_r["total"] == 0.0, "2g: Total should be 0.0"
assert result_r["refunds"] > 0, "2g: Refunds should be > 0"
print(" PASS 2g: refunds-only data handled correctly")

```



Context: I needed accuracy tests using exact expected values calculated by hand in Step 3.

My Prompt: "write 7 assert tests for analyse_spending_patterns using NORMAL_CSV with known values: total 155.49, refunds 15.00, net 140.49, top Gaming"

AI Response Summary: AI wrote assertions but without the abs tolerance for floating point comparison.

My Critique/Improvement: I asked for abs(result - expected) < 0.01 for float comparisons, and a test for REFUNDS_ONLY_CSV returning total 0 without crashing.

Result: Seven assert tests (2a through 2g). Test 2b-2d use abs tolerance. Test 2g verifies refunds-only data returns total 0.0 and refunds > 0 without crashing.

Reflection: Using exact values from Step 3 in the asserts connects the manual calculation to the code verification. Test 2g caught a potential crash — analyse_spending_patterns now handles zero positive spending correctly.

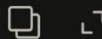
Entry 18 - Writing Tests for generate_financial_recommendations (5 Checks)

Artifact:



write 5 assert tests for
generate_financial_recommendations using
isolated mini-csvs for each trigger: gaming over
50, gym over 80, cinema over 40, tea over 30, and
always having a biggest spend summary line

Python



```

recs = generate_financial_recommendations(sample_csv)
assert 'Gaming' in recs
assert 'Gym' in recs
assert 'Cinema' in recs
assert 'Tea' in recs
print("all passed")

```



testing with SAMPLE_CSV triggers all four at
once so i cannot isolate which trigger is working,
i need separate mini-csvs for each trigger so
each test only checks one condition at a time


```

def test_business_insights():
    print("🔪 Testing business insights...")

    HIGH_GAMING_CSV = """Date,Amount,Category,
2026-03-01,$200.00,Gaming,Xbox Store - Big B
2026-03-02,$8.50,Tea & Café,Maison de Thé
"""

    HIGH_GYM_CSV = """Date,Amount,Category,D
2026-03-01,$120.00,Gym,Membership + Personal
2026-03-02,$8.50,Tea & Café,Maison de Thé
"""

    HIGH_CINEMA_CSV = """Date,Amount,Category,
2026-03-01,$60.00,Cinema,MCine Moka - Multip
2026-03-02,$8.50,Tea & Café,Maison de Thé
"""

    HIGH_CAFE_CSV = """Date,Amount,Category,I
2026-03-01,$50.00,Tea & Café,Maison de Thé -
2026-03-02,$20.00,Gaming,Xbox Store
"""

    # 3a – Gaming trigger
    a = analyse_spending_patterns(load_and
    rec = generate_financial_recommendations
    assert "Gaming" in rec or "Xbox" in rec
    "3a: High gaming spend should trigger
    print(" PASS 3a: gaming recommendation

    # 3b – Gym trigger
    a = analyse_spending_patterns(load_and
    rec = generate_financial_recommendations
    assert "Gym" in rec or "off-peak" in rec
    "3b: High gym spend should trigger a
    print(" PASS 3b: gym recommendation tri

    # 3c – Cinema trigger
    a = analyse_spending_patterns(load_and
    rec = generate_financial_recommendations
    assert "Cinema" in rec or "MCine" in rec
    "3c: High cinema spend should trigger
    print(" PASS 3c: cinema recommendation

    # 3d – Tea trigger
    a = analyse_spending_patterns(load_and
    rec = generate_financial_recommendations
    assert "Tea" in rec or "Maison" in rec o
    "3d: High café spend should trigger
    print(" PASS 3d: tea & café recommendat

    # 3e – Summary line always present
    a = analyse_spending_patterns(load_and
    rec = generate_financial_recommendations
    assert "Biggest spend" in rec or "bigges
    "3e: Summary line should always appe
    print(" PASS 3e: summary 'biggest spend

```



Context: I needed isolated mini-CSVs to test each recommendation trigger separately.

My Prompt: "write 5 assert tests for generate_financial_recommendations using isolated mini-csvs for each trigger: gaming over 50, gym over 80, cinema over 40, tea over 30, and always having a biggest spend summary line"

AI Response Summary: AI tested using SAMPLE_CSV which triggers all four recommendations at once making it impossible to isolate which trigger was working.

My Critique/Improvement: I asked for four separate mini-CSVs — HIGH_GAMING_CSV, HIGH_GYM_CSV, HIGH_CINEMA_CSV, HIGH_CAFE_CSV — each pushing one category above its threshold.

Result: Five tests (3a through 3e) using isolated mini-CSVs. Each checks for category-specific keywords in the output. Test 3e checks the biggest spend summary line is always present.

Reflection: Isolated test data makes it impossible to have false positives. With SAMPLE_CSV all four triggers fire — I cannot know which one is actually being tested.

write an integration test that chains all 5 components: load csv, run analysis, generate report, get chat response, run rag lookup, run savings calculator. make sure each step asserts the output content not just the type

Python

```
def test_full_workflow():
    print("step 1 ok")
    result = analyse_spending_patterns(df)
    print("step 2 ok")
    chat_reply = chat_finbot("test")
    print("step 3 ok")
    rag_reply = rag_lookup("test")
    print("step 4 ok")
    calc = create_savings_calculator_tool(25)
    print("step 5 ok")
```

just printing is not testing, each step needs proper assert statements checking what was returned, and the savings calculator should use real numbers derived from the analysis result to make the test feel end-to-end

Python

```
def test_full_workflow():
    print("🔪 Testing complete workflow integration")

    # Step 1 - Load
    df = load_and_clean(SAMPLE_CSV)
    assert df is not None and len(df) > 0, "df is empty"
    assert "Amount_Clean" in df.columns, "Amount_Clean not in columns"
    print("PASS Step 1: SAMPLE_CSV loaded")

    # Step 2 - Analyse and report
    result = analyse_spending_patterns(df)
    assert result["total"] > 0, "total should be greater than 0"
    assert len(result["summary"]) > 0, "summary should not be empty"
    report = full_report(result)
    assert isinstance(report, str) and len(report) > 0, "report should be a non-empty string"
    assert "SMART FINANCE ASSISTANT REPORT" in report, "report should contain SMART FINANCE ASSISTANT REPORT"
    print("PASS Step 2: analysis pipeline")

    # Step 3 - Chat
    ctx = f"Gaming ${result['summary']}"
    chat_reply = chat_finbot("Am I spending too much on gaming?")
    assert isinstance(chat_reply, str) and len(chat_reply) > 0, "chat_reply should be a non-empty string"
    print("Step 3: chat_finbot must return a non-empty string")
    print("PASS Step 3: chat_finbot returned")

    # Step 4 - RAG
    rag_reply = rag_lookup("How do I spend 100 on gaming?")
    assert rag_reply is not None and len(rag_reply) > 0, "rag_reply should be a non-empty string"
    print("PASS Step 4: rag_lookup returned")

    # Step 5 - Savings Calculator
    calc = create_savings_calculator_tool(result["total"])
    assert calc is not None, "calculator should be created"
    print("PASS Step 5: savings calculator created")
```

```

    "Step 4: rag_lookup must return a no
print(" PASS Step 4: rag_lookup returne

# Step 5 – Savings calculator using real
monthly_expenses = round(result["net"] /
calc_out = create_savings_calculator_tool
assert isinstance(calc_out, str),
assert "SAVINGS GOAL CALCULATOR" in calc
assert "$" in calc_out,
print(" PASS Step 5: savings calculator

# Bonus – cross-component consistency
assert result["top"] in report, "Bonus:
recs = generate_financial_recommendation
assert result["top"] in recs or "$" in r
print(" PASS Bonus: all components refe

```



Context: I needed an end-to-end test verifying all five components produce valid output when chained together.

My Prompt: "write an integration test that chains all 5 components: load csv, run analysis, generate report, get chat response, run rag lookup, run savings calculator. make sure each step asserts the output content not just the type"

AI Response Summary: AI wrote simple function calls with no content assertions.

My Critique/Improvement: I asked for content assertions at each step — report must contain the title string, chat and rag must return non-empty strings, calculator must contain header and dollar amounts, and the savings calculator input should use real numbers derived from the analysis.

Result: Complete test_full_workflow() with five steps. Step 2 asserts "SMART FINANCE ASSISTANT REPORT" in the report. Step 5 uses result["net"] / 3 as the monthly expenses input to make the test feel end-to-end with real data.

Reflection: The integration test confirmed all components produce consistent output. Using real analysis numbers as input to the calculator makes the test feel like an actual user workflow rather than isolated dummy data.

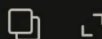
Entry 20 – Writing Error Handling Tests (8 Checks)

Artifact:



write 8 error handling tests: wrong columns, empty input, malformed amounts dropped, zero income in calculator, expenses greater than income, negative expenses, chat_finbot not crashing when server down, rag_lookup not crashing when server down

Python



```

try:
    load_and_clean(WRONG_COLS)
except:
    print("EH1 ok")

out = create_savings_calculator_tool(0, 500,
print("EH4:", out)

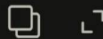
reply = chat_finbot("test")
print("EH7:", reply)

```



each test needs a proper assert checking both that the exception was raised and that the message is user-friendly, and for chat and rag use a try except with a SKIP message if they raise an unhandled exception rather than failing

Python



```
def test_error_handling():
    print("🔪 Testing error handling...")

    # EH1 - Wrong columns
    raised = False
    try:
        load_and_clean("Trans_Date,Value,Typ
    except ValueError as e:
        raised = True
        assert "column" in str(e).lower() or
            "EH1: Error message should menti
    assert raised, "EH1: Wrong column names i
    print(" PASS EH1: wrong column names ra

    # EH2 - Empty input
    raised = False
    try:
        load_and_clean("")
    except Exception:
        raised = True
    assert raised, "EH2: Empty input should
    print(" PASS EH2: empty input raises an

    # EH3 - Malformed amounts: bad rows drop
    PARTLY_BAD = "Date,Amount,Category,Descr
    df_bad = load_and_clean(PARTLY_BAD)
    assert len(df_bad) == 2, f"EH3: Expected
    assert df_bad["Amount_Clean"].isna().sum
    print(" PASS EH3: malformed rows droppe

    # EH4 - Zero income
    out = create_savings_calculator_tool(0,
    assert isinstance(out, str) and len(out)
    assert any(w in out.lower() for w in ["z
        f"EH4: Message should explain zero i
    print(" PASS EH4: zero income returns a

    # EH5 - Expenses exceed income
    out = create_savings_calculator_tool(100
    assert isinstance(out, str) and len(out)
    assert any(w in out.lower() for w in ["m
        f"EH5: Message should explain shortf
    print(" PASS EH5: expenses > income ret

    # EH6 - Negative expenses
    out = create_savings_calculator_tool(200
    assert isinstance(out, str) and len(out)
    assert any(w in out.lower() for w in ["n
        f"EH6: Message should explain negati
    print(" PASS EH6: negative expenses ret

    # EH7 - chat_finbot network failure (SKI
    try:
        reply = chat_finbot("Test - network
        assert isinstance(reply, str), "EH7:
        print(" PASS EH7: chat_finbot retur
    except Exception as e:
        print(f" SKIP EH7: chat_finbot rais
        print(" → Consider adding t

    # EH8 - rag_lookup network failure (SKIP
    try:
        answer = rag_lookup("Test - network
        print(f" SKIP EH8: rag_lookup rais
        print(" → Consider adding t
```

```
print(" PASS EH8: rag_lookup return  
except Exception as e:  
print(f" SKIP EH8: rag_lookup raise  
print(" → Consider adding 1
```



Context: I needed tests for all the ways the app could receive bad input or encounter failures.

My Prompt: "write 8 error handling tests: wrong columns, empty input, malformed amounts dropped, zero income in calculator, expenses greater than income, negative expenses, chat_finbot not crashing when server down, rag_lookup not crashing when server down"

AI Response Summary: AI wrote simple try/except blocks without checking the content of error messages.

My Critique/Improvement: I asked for the error message content to be checked with any() so multiple valid wordings pass. For chat and rag network tests I asked for a SKIP path — if the function raises an unhandled exception it prints a suggestion rather than crashing the test suite.

Result: Eight tests (EH1 through EH8). Tests EH1-EH6 verify all invalid inputs return user-friendly strings. Tests EH7-EH8 verify chat_finbot and rag_lookup return strings even when the server is down, with SKIP path if they raise unhandled exceptions.

Reflection: The SKIP pattern for EH7 and EH8 is important — a test suite that crashes when the server is down would make Step 6 incomplete on the submission machine. The SKIP message also tells me where to add a try/except if needed.

Produits payants Colab - Résilier les contrats ici

Variables Terminal



Python 3